TEAM 6
Robotic Path Planning, Inverse Kinematics and Motion Execution

Implementation-oriented team technical report

Objective: Convert detected weld geometry into feasible, collision-checked and time-parameterized robot trajectories, validate them in simulation, and provide a structured handoff to the execution team.

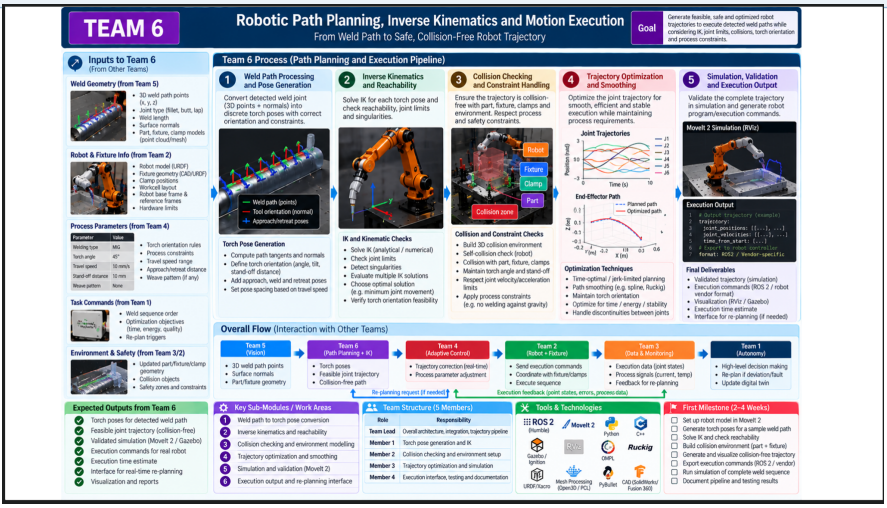


Figure 1. Team 6 workflow diagram supplied by the user: inputs, five-stage planning pipeline, cross-team flow, tool candidates and milestone checklist.

1. Problem statement and scope

Problem: Given weld-joint geometry and the robot/workcell model, generate a trajectory that follows the intended weld path while respecting available kinematic, collision and motion constraints. If a pose or trajectory is infeasible, return an explicit failure or re-planning status instead of marking it executable.

The basic mathematical and algorithmic foundations are assumed to be understood. This report focuses on practical implementation: robot descriptions, IK integration, planning-scene construction, trajectory processing, simulation, interfaces, reproducible testing and handoff.

In scope: weld waypoint processing; torch orientation and approach/retreat poses; IK and reachability; joint limits and singularity indicators; self/environment collision checking; trajectory planning and smoothing; simulation validation; trajectory export and re-planning interface.

Dependencies: Team 5 supplies weld geometry and confidence; Team 2 supplies robot/fixture/clamp models and execution interface; Team 4 supplies process constraints and correction requests; Team 1 supplies weld order/optimization priorities; Team 3 may provide process/environment feedback.

Status note: this is a proposed implementation plan. The supplied diagram does not prove that a particular robot, solver, collision model or successful simulation has already been configured.

2. Proposed workflow and tool selection

Stage	Implementation work	Candidate tools / output
1. Weld path and torch poses	Validate joint geometry; resample points; estimate local tangent/normal; define tool orientation from an agreed process rule; create approach, weld and retreat poses.	Python/NumPy prototype; agreed TCP/frame. Output: Cartesian poses.
2. IK and reachability	Solve IK for each pose; preserve continuity across waypoints; reject unreachable points; check joint limits and a supported singularity indicator.	MoveIt 2 kinematics plugin or robot-specific solver. Output: candidate joint states/failure.
3. Collision and constraints	Model robot, part, fixture, clamps and relevant scene geometry; check self/environment collision and configured motion/process limits.	MoveIt planning scene and collision checker. Output: status/reason.
4. Planning, timing and smoothing	Use collision-aware planner for approach/retreat/transfers; time-parameterize; optionally apply jerk-limited smoothing; validate the final trajectory again.	MoveIt 2 + OMPL candidate; Ruckig optional. Output: timed trajectory.
5. Simulation and export	Visualize full sequence, estimate duration, export agreed representation and return re-plan/fault status when required.	RViz for visualization; Gazebo Sim or project-approved simulator for physics/sensors if needed.

Preferred technology stack

Need	Candidate tool	Reason / caveat
Robot & workcell	URDF/Xacro, collision meshes; CAD supplied by partners	Must match actual robot geometry, TCP, frames and limits; verify scale/orientation.
Motion planning	MoveIt 2 + OMPL if ROS 2 is agreed	Planning-scene representation, plugins and collision checking; benchmark planner and settings.
IK	Robot-compatible MoveIt solver/plugin	Select only after robot model and joint ranges are confirmed.
Trajectory timing	MoveIt time parameterization; Ruckig optional	Configured velocity/acceleration and optional jerk limits; recheck final path.
Visualization/simulation	RViz + Gazebo Sim or existing simulator	RViz visualizes plans; Gazebo Sim provides physics/rendering/sensor models.
Development/testing	C++/ROS 2 integration, Python scripts/metrics, pytest or repo-standard tests	Fit existing repository and versions; avoid parallel infrastructure.
Geometry utilities	Open3D/PCL only if mesh/point-cloud conversion is needed	Team 6 consumes Team 5 geometry rather than duplicating perception.

Alternatives considered

Alternative	Decision
Custom IK and full planner	Not the first choice: larger implementation and test burden; revisit only if existing framework cannot meet a specified need.
Cartesian interpolation alone	Useful to follow weld geometry but not enough for collision-free transfer or guaranteed joint feasibility; validate through IK/planning.
MoveIt/OMPL	Preferred general motion-planning candidate; benchmark settings on actual robot model and scene.
Ruckig smoothing	Optional jerk-limited post-processing; confirm compatibility and recheck collision/path deviation afterward.
PyBullet instead of Gazebo	Possible standalone prototype alternative; avoid maintaining two simulators without a clear test need.

Baseline: scene → torch poses → IK → collision-aware path → time parameterization → final validation → simulation → trajectory JSON. Add custom algorithms only when baseline tests show a specific gap.

3. Interfaces and input contract

Producer	Input to Team 6	Purpose
Team 5 — Vision	Joint ID/type, 3D points, normals, length/confidence, part/fixture/clamp geometry, frame and timestamp.	Create torch poses and update collision scene.
Team 2 — Robot/cell	Robot description, TCP, joint ranges, fixture/clamp poses, workcell layout and controller capabilities.	Configure IK, collision checks and eventual execution format.
Team 4 — Adaptive/process	Process type, torch angle/stand-off/speed constraints and agreed correction request.	Apply process constraints and trigger re-planning when needed.
Team 1 — Autonomy	Weld order, optimization priorities and re-plan trigger.	Plan one joint or an ordered sequence.
Team 2 — Execution	Consumes validated trajectory/status; provides completion/fault feedback.	Execute only through approved controller interface.

Input example: scene.json (mock payload)

Units: metres, radians and seconds. Timestamp: ISO 8601 UTC. Positions are expressed in robot_base. Quaternion order is [x, y, z, w]. Mesh paths/numbers are illustrative, not measured hardware data.


```
{
 "schema_version": "0.1", "scene_id": "scene_mock_001",
 "timestamp": "2026-10-19T10:00:00Z", "coordinate_frame": "robot_base",
 "units": {"distance":"m", "angle":"rad", "time":"s"},
 "robot": {"robot_id":"robot_mock_01", "model_uri":"models/robot.urdf", "tcp_frame":"welding_tcp"},
 "part": {"id":"part_001", "mesh_uri":"meshes/part.stl", "position_m":[0.45,0,0.20], "orientation_xyzw":[0,0,0,1]},
 "fixtures": [{"id":"fixture_001", "mesh_uri":"meshes/fixture.stl", "position_m":[0.45,0,0.10], "orientation_xyzw":[0,0,0,1]}],
 "clamps": [{"id":"clamp_001", "mesh_uri":"meshes/clamp.stl", "position_m":[0.40,0.10,0.16], "orientation_xyzw":[0,0,0,1]}],
 "weld_joints": [{"id":"joint_001", "type":"fillet", "start_m":[0.35,0.02,0.22],
   "end_m":[0.55,0.02,0.22], "normal":[0,0,1], "length_m":0.20, "confidence":0.93, "validity":"MOCK_VALID"}]
}
```


Key input fields

Field	Meaning / contract rule
schema_version, scene_id, timestamp	Required version, unique scene ID and capture time; reject/flag stale inputs.
coordinate_frame, units	Required. All positions use the same declared frame; use m/rad/s internally.
robot.model_uri, tcp_frame	Robot description and tool frame; placeholders must be replaced before robot-specific checks.
part / fixtures / clamps	Unique IDs, resolvable collision mesh and pose; validate mesh scale and orientation.
weld_joints[]	Unique ID; type when known; start/end/normal/length; confidence [0,1] and validity. Team 5 owns estimates.

Requires team agreement: robot_base vs world frame, timestamp-age limit, confidence threshold, quaternion convention, mesh path convention and whether planner output uses Cartesian poses, joint points or both.

4. Output contract and planner-to-execution interface

Output example: trajectory.json (mock candidate, not executed)

This is a valid illustrative payload. Joint values are not claimed to be solved for any particular robot. Validation states are deliberately NOT_RUN until checks actually execute.


```
{
 "schema_version":"0.1", "trajectory_id":"traj_mock_001",
 "timestamp":"2026-10-19T10:01:00Z", "robot_id":"robot_mock_01",
 "coordinate_frame":"robot_base", "units":{"position":"m","joint_position":"rad","time":"s"},
 "status":"PLANNED_PENDING_VALIDATION",
 "joint_names":["joint_1","joint_2","joint_3","joint_4","joint_5","joint_6"],
 "waypoints":[
  {"time_from_start_s":0.0,"joint_positions_rad":[0.0,-0.5,0.8,0.0,0.6,0.0],
   "tcp_position_m":[0.35,0.02,0.24],"tcp_orientation_xyzw":[0,0,0,1],"phase":"APPROACH"},
  {"time_from_start_s":1.0,"joint_positions_rad":[0.1,-0.45,0.75,0.0,0.65,0.0],
   "tcp_position_m":[0.45,0.02,0.22],"tcp_orientation_xyzw":[0,0,0,1],"phase":"WELD"},
  {"time_from_start_s":2.0,"joint_positions_rad":[0.15,-0.4,0.7,0.0,0.7,0.0],
   "tcp_position_m":[0.55,0.02,0.22],"tcp_orientation_xyzw":[0,0,0,1],"phase":"RETREAT"}],
 "validation":{"ik":"NOT_RUN","joint_limits":"NOT_RUN","singularity":"NOT_RUN",
   "collision":"NOT_RUN","velocity_acceleration":"NOT_RUN","torque":"NOT_AVAILABLE"},
 "next_action":"VALIDATE_BEFORE_EXECUTION",
 "notes":["Mock payload only; not from a real robot or completed simulation."]
}
```


Output field definitions

Field	Meaning / rule
trajectory_id, timestamp, robot_id	Unique trajectory, generation time and intended robot.
coordinate_frame, units, joint_names	Frame, SI units and explicit joint ordering required by the controller.
time_from_start_s	Monotonic time from trajectory start in seconds.
joint_positions_rad	Joint-space positions in radians in joint_names order.
TCP position/orientation	Pose for validation/visualization; quaternion order [x,y,z,w].
phase	APPROACH, WELD, TRANSFER or RETREAT; exact enum needs agreement.
validation/status	Each check is PASS, FAIL, NOT_RUN or NOT_AVAILABLE. Do not call trajectory executable before required checks pass.
next_action	Execution, rejection, hold or replan. Team 2 supplies completion/fault acknowledgement.

Integration and safety

Team 6 receives scene.json from Team 5, robot/workcell/process constraints from Teams 2/4, and sequence requests from Team 1. It sends trajectory.json and validation status to Team 2 for execution and Team 1 for decision support. Team 4 correction or new Team 5 geometry may trigger replanning. Physical execution still requires approval through the responsible robot/cell safety process; a passing simulation is not certification of industrial safety or weld quality.

5. Baseline, validation and risks

Baseline implementation without neural algorithms

- Use a documented robot model or labelled simulated substitute, one part, fixture, clamps and one weld joint.
- Convert 3D joint points/normals into discrete torch poses, including approach, weld and retreat using the process team rules.
- Solve IK at each pose, check joint bounds and continuity, and reject unreachable points.
- Build planning scene; test self-collision and robot collision against part, fixture and clamps.
- Plan free-space transfers, time-parameterize/smooth if supported, then recheck collision and weld-path deviation.
- Visualize trajectory; export a candidate JSON plus planner settings, validation results and run logs.

Required scenarios

Scenario	Type	Expected evidence
Single straight reachable weld with clear fixture	Normal	Torch poses, IK result, candidate trajectory, duration and JSON.
Curved seam / multiple sampled points	Normal	Pose/waypoint continuity and path-deviation measurement.
Multiple joints with clamp geometry	Normal	Sequence and inter-joint transfer validated in scene.
Pose outside workspace	Failure	IK_FAILED/UNREACHABLE and no executable output.
Route intersects clamp/part	Failure	Collision failure or alternate valid path; log the result.
Joint limit/singularity/invalid mesh or frame	Failure	Correct failure category or NOT_AVAILABLE; hold/replan response.

Evaluation metrics

Metric	Definition
Reachable-pose rate	Required torch poses with valid IK / total required poses.
Collision-free trajectory rate	Complete candidates passing supported collision checks / total attempts.
Planning time	Wall-clock time from planning request to response; state model, scene, planner and machine.
Path deviation	Distance between planned TCP samples and desired seam points; report mean, 95th percentile and max.
Smoothness / limits	Declared velocity/acceleration/jerk measures and violation counts where model data supports them.
Sequence duration	Total of approach, weld, retreat and transfer durations.
Replanning success	Fraction of changed/failed requests for which a feasible revised trajectory is found.

Risks and fallback

Risk	Mitigation
Robot URDF/TCP/model unknown	Use documented substitute and mark robot-specific claims pending.
Poor mesh or incorrect scale	Validate geometry dimensions and transforms before collision tests.
IK solutions jump between points	Check joint-space continuity and choose among supported solutions.
Smoothing alters path	Recheck collision and weld-path deviation after post-processing.
No physical robot/safety approval	Deliver software/simulation baseline only; do not claim physical execution.
Stack becomes too complex	Start with project tools and MoveIt/OMPL candidate; add PyBullet/custom planner only if justified.

Set acceptance thresholds with the project lead after robot accuracy, sensing uncertainty and process tolerances are known. The workflow image is a proposal; its illustrative curves and values are not measured results.